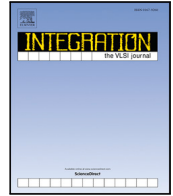




Contents lists available at ScienceDirect

Integration, the VLSI Journal

journal homepage: www.elsevier.com/locate/vlsi



An efficient architecture of truncated booth multiplier for AI application

Shareefa Fairoose P.¹, Ashutosh Mishra^{1,*},²

Department of ECE, National Institute of Technology Calicut, India

ARTICLE INFO

Keywords:

Truncated approximate booth multiplier
Approximate Partial Product Generator
Approximate Compressor
Leading One/Zero Position Detector

ABSTRACT

This paper presents truncated approximate booth multipliers (TR-ABMs) that are both energy- and area-efficient, leveraging novel Leading One/Zero Position Detectors (LOZPDs) and optimized approximate booth multiplier (ABM) architectures. The proposed LOZPDs enable substantial reductions in Area-Delay Product (ADP) and Power-Delay Product (PDP) relative to existing techniques. Two architectures are introduced: TR-ABM1 integrates LOZPD-based operand truncation with a conventional Booth multiplier, while TR-ABM2 employs an approximate Booth multiplier variant for further efficiency gains. The level of approximation is tunable through key design parameters, including multiplier width (w) and the number of partial product columns utilizing Approximate Partial Product Generators (APGs) and Approximate Compressors (ACs). Comprehensive error analysis is conducted via Monte Carlo simulations with 10 million random inputs, and the designs are synthesized using Cadence® Genus in 90 nm CMOS technology. The TR-ABMs are evaluated in neural network (NN) inference and 64-point Fast Fourier Transform (FFT64) applications. For MNIST handwritten digit classification, the TR-ABMs achieve accuracy on par with exact fixed-point Booth multipliers. In FFT64, the proposed designs deliver significant area and power savings over state-of-the-art approximate multipliers. Specifically, the TR-ABMs achieve 66.58%–75.59% reductions in ADP and 47.91%–60.94% reductions in PDP, while maintaining reliable computational accuracy. Overall, the TR-ABMs offer a superior accuracy-performance trade-off compared to prior approximate multipliers, making them highly suitable for energy-efficient artificial intelligence and signal processing applications.

1. Introduction

Approximate computing has emerged as a promising paradigm for enhancing the energy efficiency of error-tolerant applications, including digital signal processing (DSP), multimedia, and machine learning. Among the various arithmetic operations, multiplication is particularly power-intensive and has therefore been the subject of extensive research. Approximate multipliers offer an attractive solution by trading a marginal loss in accuracy for substantial reductions in power consumption, silicon area, and computational delay, making them highly suitable for these application domains.

A conventional multiplier architecture typically comprises two main stages: Partial Product Generation (PPG) and Partial Product Accumulation (PPA). Numerous approximation techniques have been investigated for both stages. In the PPG stage, researchers have widely explored strategies such as reducing the number of partial product rows or employing approximate Booth encoding [1–4].

Several Approximate Booth Multipliers (ABMs) have been proposed in the literature to achieve improved computational efficiency and

reduced hardware complexity [2–14]. For instance, the designs in [4,6] integrate an Approximate Booth Encoder (ABE) in the PPG stage and Approximate Compressors (ACs) in the PPA stage, introducing complementary negative and positive errors that effectively cancel each other, thereby minimizing the Mean Relative Error (MRE). Venkatachalam et al. [2] proposed three ABM variants based on ABE and partial product truncation, while Liu et al. [15] introduced two ABEs and a corresponding ABM architecture.

Partial product truncation has been extensively employed in ABM designs as a means of reducing power and area overheads, often combined with compensation mechanisms to mitigate the truncation error. For example, the work in [16] utilizes probabilistic analysis of Booth-encoded partial products; a dual-mode truncation error compensation technique is proposed in [17]; [18] derives compensation values based on sign-bit conditional probabilities; and [19] applies scaling with redundant bits. While such approaches have demonstrated effectiveness for error-tolerant applications, they often incur large relative errors when operands are small. This limitation arises because least significant

* Corresponding author.

E-mail addresses: shareefa_p210133ec@nitc.ac.in (Shareefa Fairoose P.), ashutosh@nitc.ac.in (A. Mishra).

¹ Student.

² Assistant professor.

bits are either discarded – with only a compensation term added to the most significant region – or ABM-based architectures are used using approximate logic.

Leading One Detector (LOD)-based approximate multipliers offer an alternative to partial-product modification by approximating directly at the operand level. They achieve these objectives by dynamically truncating inputs according to the position of the most significant bit [20], thereby reducing the partial product array, lowering power consumption, and improving accuracy for small inputs compared to array-based multipliers. Notable examples include Mitchell's logarithmic multiplier [21], LOD-enhanced variants [22], and truncated-operand schemes such as TOSAM [23–27]. These approaches adjust operand width based on the leading-one position. For instance, the method in [27] truncates operands to m bits defined by the leading one, but the result consistently underestimates the exact product, producing a negative Mean Relative Error (MRE). To address this bias, [24] forces the least significant truncated bit to 1, effectively steering the MRE toward zero.

Conventional LOD-based multipliers provide only limited support for signed multiplication, although many target applications—such as neural networks, FFT, and image processing require full signed-operation capability. Standard LODs detect only the first '1' in an unsigned input, which necessitates preprocessing to map two's complement operands into unsigned form and post-processing to recover the signed result [23,24,26]. These additional stages, often implemented with large numbers of XOR gates, introduce significant switching activity, thereby increasing area, delay, and dynamic power. Moreover, for negative numbers in two's complement representation, the most significant position is indicated by the leading zero, which conventional LODs are not inherently capable of detecting.

Yuxuan [5] addressed part of this limitation by proposing a Leading One/Zero Detector (LOZD) capable of identifying either the leading '1' or '0'. However, the design still required a separate signed encoder stage to determine the bit position, adding to circuit complexity.

To overcome these limitations, we propose the fused Leading One/Zero Position Detector (LOZPD) specifically designed for signed multipliers. Unlike prior methods that rely on separate preprocessing and encoding, the proposed LOZPD directly identifies and encodes the position of the first '1' in positive operands and the first '0' in negative operands. This direct detection and encoding eliminates redundant circuitry and achieves significantly lower Area-Delay Product (ADP) and Power-Delay Product (PDP) compared to existing leading-bit detection schemes. In addition, by allowing the accurate multiplier within the truncated structure to be replaced with an approximate multiplier, the design delivers further reductions in area, power, and delay. Collectively, these innovations establish the proposed LOZPD as a highly efficient and scalable solution for signed approximate multipliers, overcoming the fundamental limitations of conventional LOD and LOZD-based designs.

The primary contributions of this research include:

1. A fused Leading One/Zero Position Detector (LOZPD) for signed multipliers that directly identifies the leading '0' in negative operands and the leading '1' in positive operands, eliminating the need for preprocessing, post-processing, and encoding stages, and delivering significantly lower Area-Delay Product (ADP) and Power-Delay Product (PDP) than conventional LOD- and LOZD-based designs.
2. An efficient multiplier architecture that integrates the proposed LOZPD with a tunable approximation parameter w , enabling a configurable trade-off between accuracy and energy efficiency, and achieving substantial ADP and PDP reductions compared to state-of-the-art multipliers.
3. A novel approximation strategy for truncated multipliers that substitutes the accurate Booth multiplier with a low-error approximate Booth multiplier featuring tunable approximation levels, thereby achieving further reductions in delay, power, and area with minimal accuracy degradation.

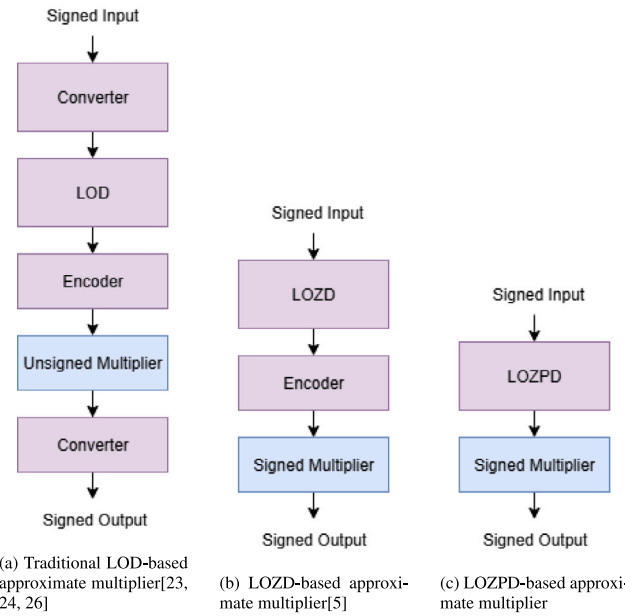


Fig. 1. Signed truncated multiplier architectures using LOD, LOZD, and Proposed LOZPD.

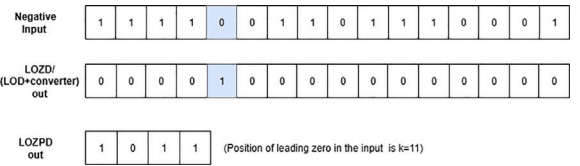


Fig. 2. Output of LOZD/(Converter+LOD) and LOZPD for negative input.

The remainder of the paper is organized as follows: Section 2 introduces the proposed LOZPD and TR-ABM architectures. Section 3 presents the error analysis and VLSI implementation results. Section 4 reports the application-level performance evaluation, and Section 5 concludes the paper.

2. Design of proposed LOZPD and truncated booth multiplier

This section details the design of the proposed area, power, and delay-efficient LOZPD for operand truncation, suitable for integration into any signed multiplier. It further introduces the design of an efficient Truncated Approximate Booth Multiplier (TR-ABM1) that incorporates the proposed LOZPD.

2.1. Leading One/Zero Position Detector (LOZPD)

Fig. 1(a)–(c) depict the signed multiplier design flows based on LOD, LOZD, and the proposed LOZPD architectures, respectively. The LOD-based flow (Fig. 1(a)) requires both preprocessing and post-processing converters, implemented as XOR gate arrays, where the number of XOR gates equals the operand bit-width. These converters transform negative operands into their absolute values before multiplication and restore the correct sign at the output. The LOZD-based flow (Fig. 1(b)) eliminates these converters, while the proposed LOZPD-based flow (Fig. 1(c)) further removes the need for both converters and encoders. In other words, an LOZD circuit is functionally equivalent to Converter + LOD, while an LOZPD circuit is equivalent to Converter + LOD + Encoder, as shown in Fig. 1.

Fig. 2 illustrates the outputs of (Converter + LOD/LOZD) and LOZPD for a negative operand. In (Converter + LOD/LOZD), only the

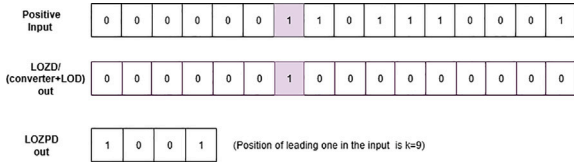


Fig. 3. Output of LOZD/(Converter+LOD) and LOZPD for positive input.

Table 1

Truth table of LOPD.

Input				Output		Zero
x_3	x_2	x_1	x_0	y_1	y_0	V
0	0	0	0	0	0	1
0	0	0	1	0	0	0
0	0	1	X	0	1	0
0	1	X	X	1	0	0
1	X	X	X	1	1	0

Table 2

Truth table of LZPD.

Input				Output		One
x_3	x_2	x_1	x_0	p_1	p_0	P
1	1	1	1	0	0	1
1	1	1	0	0	0	0
1	1	0	X	0	1	0
1	0	X	X	1	0	0
0	X	X	X	1	1	0

detected bit position is asserted '1', while all others remain 0, requiring an additional encoding stage to generate the binary index for operand truncation and shift computation. In contrast, LOZPD directly outputs this binary index. For example, when the leading zero occurs at position $k = 11$ from the LSB, LOZPD produces the output 1011. Similarly, Fig. 3 shows the case of a positive operand, where the leading '1' is at position $k = 9$ from the LSB, yielding the LOZPD output 1001.

New architectures for 4, 8, 16, and 32-bit LOZPDs are presented, with scalability to any power-of-two bit width. Notably, the proposed LOZPDs are exact designs with no approximations, ensuring fully accurate outputs. The detailed circuit implementations are discussed in the following section.

2.2. Proposed 4-bit LOZPD

Tables 1 and 2 present the truth tables of the LOPD and LZPD, respectively, with binary-indexed outputs. For the LOPD, an output of $V = 1$ denotes that no leading one is present in the input, whereas for the LZPD, an output of $P = 1$ denotes that no leading zero is present. From Table 1, outputs of LOPD are obtained as follows:

$$y_1 = x_3 + x_2 \quad (1)$$

$$y_0 = x_3 + \overline{x_2} \cdot x_1 \quad (2)$$

$$V = \overline{x_3} \cdot \overline{x_2} \cdot \overline{x_1} \cdot \overline{x_0} \quad (3)$$

From Table 2, outputs of LZPD are obtained as follows:

$$p_1 = \overline{x_3} + \overline{x_2} \quad (4)$$

$$p_0 = \overline{x_3} + x_2 \cdot \overline{x_1} \quad (5)$$

$$P = x_3 \cdot x_2 \cdot x_1 \cdot x_0 \quad (6)$$

Eqs. (1)–(6) show that the LZPD and LOPD share the same underlying circuitry, differing only in their input form. Specifically, an LOPD

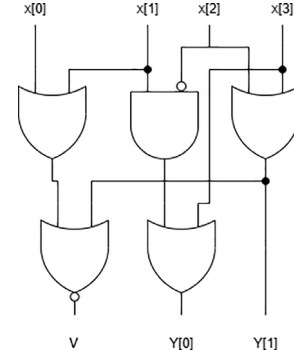


Fig. 4. Circuit for LOPD using Eqs. (1)–(3).

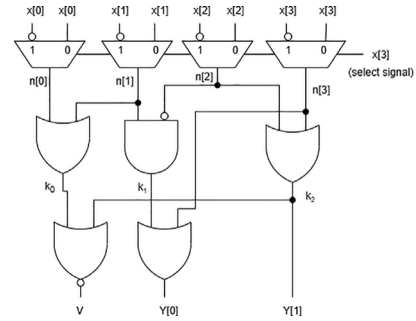


Fig. 5. 4-bit LOZPD circuit derived from LOPD and LZPD.

Table 3

Truth table of proposed 4-bit LOZPD.

Input				Output		
x_3	x_2	x_1	x_0	y_1	y_0	V
0	0	0	0	0	0	1
0	0	0	1	0	0	0
0	0	1	X	0	1	0
0	1	X	X	1	0	0
1	0	X	X	1	0	0
1	1	0	X	0	1	0
1	1	1	0	0	0	0
1	1	1	1	0	0	1

with inputs (x_3, x_2, x_1, x_0) can operate as an LZPD when all inputs are complemented. This property facilitates the design of a unified LOZPD architecture, where fusion can be achieved by selecting either complemented or uncomplemented inputs based on the Most Significant Bit (MSB).

The LOPD and the unified LOZPD architectures are shown in Fig. 4 and Fig. 5, respectively. In Fig. 5, the input signals pass through a 2:1 multiplexer, with the MSB as the select line, enabling the selection between complemented and uncomplemented inputs.

Table 3 presents the truth table of the proposed 4-bit LOZPD, illustrating its ability to dynamically switch between LOPD and LZPD modes depending on the MSB. Higher bit-width LOZPDs are derived directly from this 4-bit design.

Using Boolean algebra, the design for LOZPD can be further optimized for a compact circuit as shown in Fig. 6. The simplification process is shown in Eqs. (7)–(11). This eliminates the redundant logic while preserving functionality.

$$n[3] = \overline{x[3]} \cdot x[3] + x[3] \cdot \overline{x[3]} = 0 \quad (7)$$

$$y[1] = n[3] + n[2] = n[2] \quad (8)$$

$$y[0] = k_1 + n[3] = k_1 \quad (9)$$

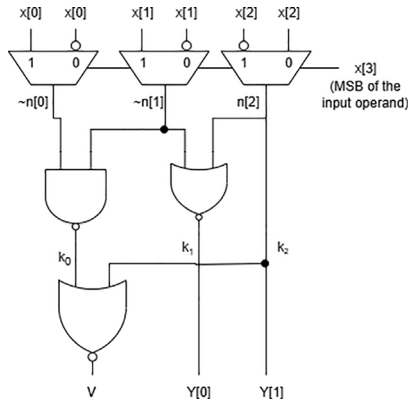


Fig. 6. Proposed 4-bit LOZPD/MSB_LOZPD.

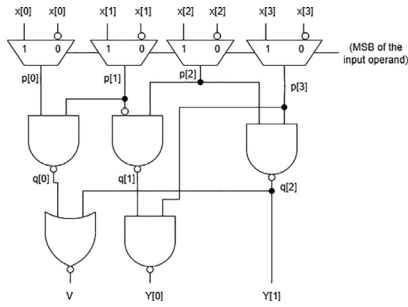


Fig. 7. An Optimized Intermediate LOZPD design.

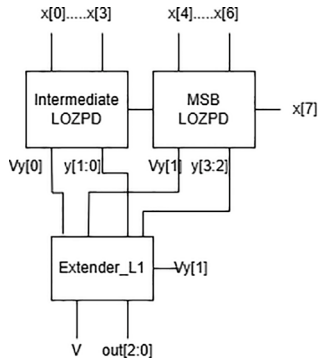


Fig. 8. 8-bit LOZPD.

$$k_1 = y[0] = n[1] \cdot \overline{n[2]} = \overline{n[2]} + n[1] \quad (10)$$

$$k_0 = n[0] + n[1] = \overline{n[0]} \cdot \overline{n[1]} \quad (11)$$

2.3. Proposed 8-bit, 16-bit and 32-bit LOZPD

In the design of 8-bit, 16-bit, and 32-bit LOZPDs, the 4-bit LOZPD serves as the fundamental building block. Accordingly, the input is divided into groups of 4 bits for processing. The most significant group is processed using the optimized 4-bit LOZPD shown in Fig. 6, hereafter referred to as the MSB LOZPD, while the remaining groups are handled by the architecture depicted in Fig. 7, hereafter referred to as the Intermediate LOZPD. Intermediate LOZPD is an optimized architecture obtained from the circuit shown in Fig. 5 using NAND/NOR replacement. The MSB group and the remaining groups are processed by two

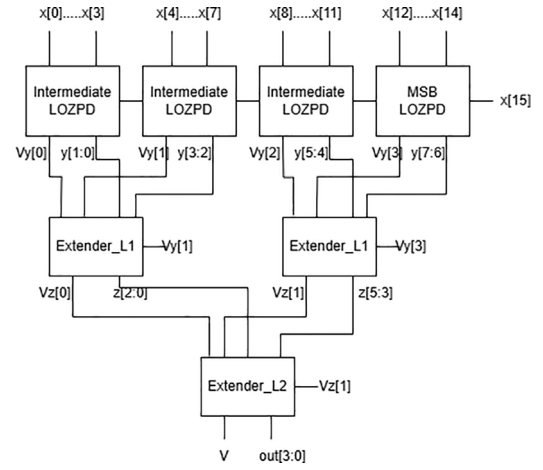


Fig. 9. 16-bit LOZPD.

different architectures for 4-bit LOZPDs because the most significant bit of the input is working as a select line.

The MSB-LOZPD and each Intermediate-LOZPD map a 4-bit input to a 3-bit output, which is subsequently processed by extenders. For an 8-bit input, a single extender *Extender_L1* is required. In case of a 16-bit input, two extenders *Extender_L1* and *Extender_L2* are used, while a 32-bit input requires three extenders *Extender_L1*, *Extender_L2*, and *Extender_L3*.

The architecture of the proposed 8-bit, 16-bit, and 32-bit LOZPDs are illustrated in Figs. 8, 9, and 10.

2.4. Extender for 8-bit, 16-bit and 32-bit LOZPD

To illustrate the design process, consider the 8-bit LOZPD shown in Fig. 8. The *Extender_L1* block, depicted in Fig. 11, receives a total of six input bits—three bits each from the outputs of two 4-bit LOZPDs.

- The *MSB_LOZPD* processes the input segment $x[7 : 4]$ and generates outputs $V[1], Y[3 : 2]$.
- The *Intermediate LOZPD* processes the input segment $x[3 : 0]$ and generates outputs $V[0], Y[1 : 0]$.

These signals serve as the inputs to *Extender_L1*. The resulting output, denoted as V and $out[3 : 0]$, can be expressed as follows:

$$V = x[7] \cdot x[6] \cdot x[5] \cdot x[4] \cdot x[3] \cdot x[2] \cdot x[1] \cdot x[0] = V[0] \cdot V[1]$$

where,

$$V[0] = x[3] \cdot x[2] \cdot x[1] \cdot x[0], \quad V[1] = x[7] \cdot x[6] \cdot x[5] \cdot x[4]$$

The Leading One/Zero (LO/LZ) position is determined by analyzing two cases:

Case 1: *MSB_LOZPD* contains a LO/LZ (ie, $V[1] = 0$).

In this case, the output of the *MSB_LOZPD* is selected as $out[1:0]$. Since the result lies in the upper range $[4:7]$, $out[2]$ is explicitly set high:

$$V[1] = 0 \Rightarrow out[1 : 0] = Y[3 : 2], \quad out[2] = 1.$$

Case 2: *MSB_LOZPD* not contains a LO/LZ (ie, $V[1] = 1$).

Here, the output of the *Intermediate_LOZPD* is selected as $out[1:0]$. Since the LO/LZ lies within the lower range $[0:3]$, $out[2]$ is set low:

$$V[1] = 1 \Rightarrow out[1 : 0] = Y[1 : 0], \quad out[2] = 0.$$

Thus, the selection between the *MSB_LOZPD* and *Intermediate_LOZPD* outputs is realized using a 2:1 multiplexer, as shown in Fig. 11, and

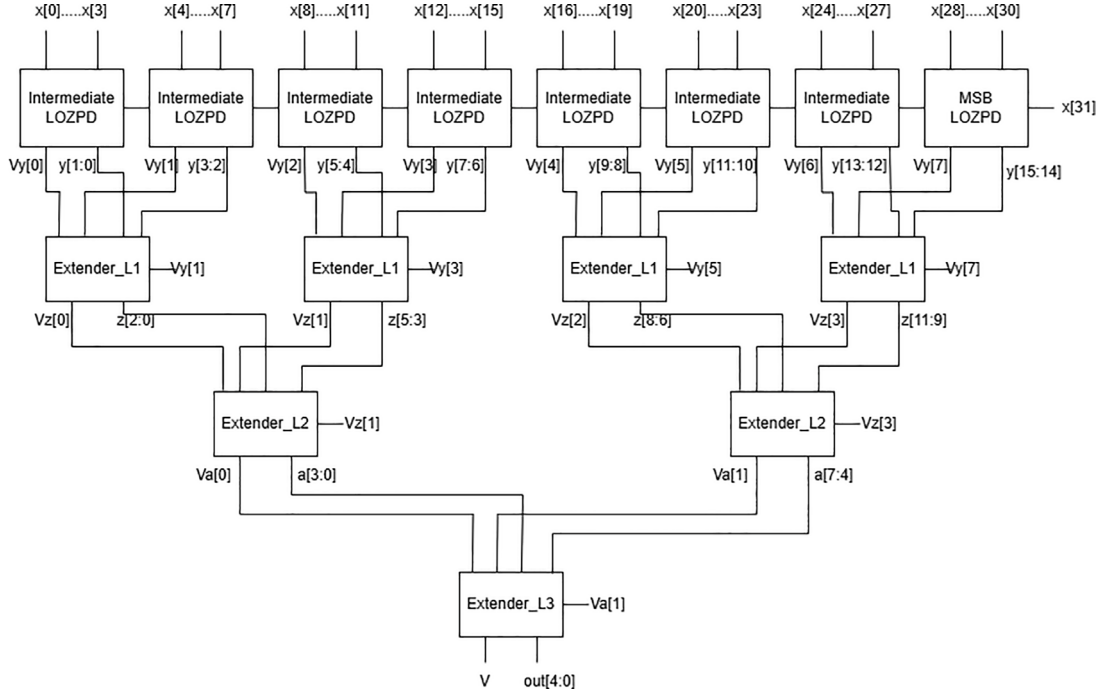


Fig. 10. Proposed 32-bit LOZPD.

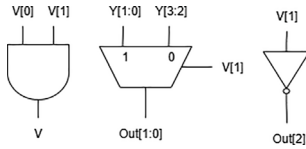


Fig. 11. Extender_L1.

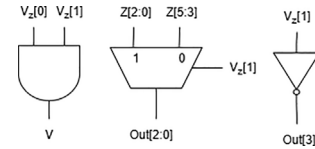


Fig. 12. Extender_L2.

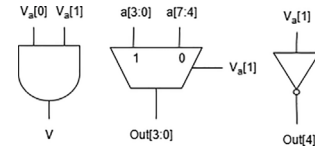


Fig. 13. Extender_L3.

$V[1]$ serves as the select line for MUX. Moreover, it follows directly that

$$\text{out}[2] = \overline{V[1]}.$$

The architectures of *Extender_L2* and *Extender_L3* are direct extensions of *Extender_L1*. Specifically, *Extender_L2* integrates the 8-bit output generated by two *Extender_L1* units (each producing a 4-bit output), while *Extender_L3* combines the 10-bit outputs from two *Extender_L2* units (each producing a 5-bit output). In both cases, the selection logic prioritizes the MSB extender output when a valid LO/LZ is present; otherwise, the LSB extender output is selected. This hierarchical organization of extenders, constructed from 4-bit LOZPD modules, enables scalable implementations of LOZPDs with 16-, 32-, or higher bit-widths. Figs. 12 and 13 represents the architecture of *Extender_L2* and *Extender_L3*, respectively. One can observe that all extenders are following the same architectural design, and the only difference is in their input and output bit width. The proposed 8-bit and 16-bit LOZPDs were synthesized using Cadence® Genus with a 90 nm CMOS technology, alongside previously reported designs for comparison. The TOSAM design [23], originally intended for unsigned inputs, was adapted for signed inputs by integrating the preprocessing module as described in [23]. The synthesis results, summarized in Table 4, demonstrate that the proposed LOZPD achieves significant improvements in Area-Delay Product (ADP) and Power-Delay Product (PDP) compared to state-of-the-art solutions.

Table 4

Synthesis results of LOZPDs.

LOZPD	Area (μm^2)	Power (nW)	Delay (ps)	ADP ($\mu\text{m}^2\text{ns}$)	PDP (nW μs)
8-bit LOZPD					
TOSAM [23]	177.87	5084.51	519	92.31	2.638
Yuxuan [5]	177.87	4384.367	524	93.2	2.297
Proposed	96.88	2767.497	220	21.31	0.608
16-bit LOZPD					
TOSAM [23]	480.63	12537.7	1122	539.27	14.067
Yuxuan [5]	429.919	10020.07	1141	490.53	11.432
Proposed	227.07	5527.034	392	89.011	2.166

2.5. Proposed truncated approximate booth Multiplier1 (TR-ABM1)

The proposed Truncated Approximate Booth Multiplier (TR-ABM1) architecture, along with an example, is depicted in Fig. 14. Its corresponding algorithmic description is outlined in Algorithm 1. The design introduces a novel combination of dynamic operand truncation

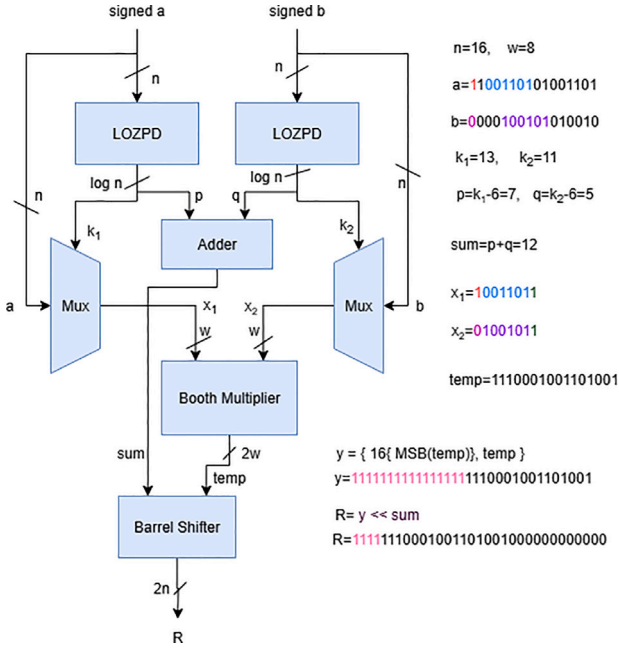


Fig. 14. Proposed Truncated Booth Multiplier(TR-ABM1).

and conventional Booth multiplication. It is organized into four stages consisting of leading position detection, adaptive truncation, Booth multiplication, and result scaling.

The signed inputs a and b are first processed by dedicated Leading One/Zero Position Detectors (LOZPDs). These modules identify the position of the most significant non-redundant bit (leading one or zero) of each operand, denoted by k_1 and k_2 , which subsequently guide the truncation, multiplication, and scaling process.

For example, as shown in Fig. 14

$a = 1100110101001101$ and $b = 0000100101010010$ the LOZPD yields $k_1 = 13$ (leading zero in a) and $k_2 = 11$ (leading one in b).

The adaptive truncation stage utilizes multiplexers (denoted as Mux in Fig. 14) to dynamically select w -bit segments from each operand, where w is the configurable width of the Booth multiplier. In Algorithm 1, this functionality is implemented by the Mux submodule, whose operation is outlined in Algorithm 2. The mux follows two selection modes based on the detected leading position of the input operand:

Case 1: If $select \geq w - 1$ (i.e., k_1 or $k_2 \geq w - 1$), the operand contains more than w valid bits. The multiplexer then selects the $w - 2$ most significant bits starting from the detected leading position, prefixes the original sign bit at the MSB, and appends a logic 1'b1 at the LSB. The additional LSB '1' serves as an error-compensation term, mitigating the negative bias commonly introduced by truncation [5,24].

Case 2: If $select < w - 1$, the operand contains fewer than w valid bits. In this case, the multiplexer directly outputs the w least significant bits without truncation, and the Booth multiplier yields an exact product. Thus, TR-ABM1 produces accurate results whenever both operands satisfy $k_1, k_2 < w - 1$.

Through this operand truncation mechanism, the resulting operands (x_1 and x_2 in the illustrative example) preserve the essential magnitude information of the original inputs while reducing computational complexity. In the example shown in Fig. 14, both k_1 and k_2 exceed $w - 1$, with values of $k_1 = 13$ and $k_2 = 11$, while w is equal to 8. The truncated operands are therefore expressed as

$$\begin{aligned} x_1 &= \{msb(a), a[k_1 : w - 2], 1'b1\} \\ &= \{1'b1, a[13 : 6], 1'b1\} = 10011011 \end{aligned}$$

Algorithm 1: TR-ABM

Input: a, b : n -bit signed inputs

Input: Let w : input bit width of Booth multiplier

Output: R : $2n$ -bit signed output

```

 $k_1 \leftarrow LOZPD(a);$  // Find LOP/LZP of  $a$ 
 $k_2 \leftarrow LOZPD(b);$  // Find LOP/LZP of  $b$ 
 $i_1 \leftarrow mux(k_1, a);$  // Select  $w-2$  bits from  $a$ 
 $i_2 \leftarrow mux(k_2, b);$  // Select  $w-2$  bits from  $b$ 
 $m_1 \leftarrow msb(a);$   $m_2 \leftarrow msb(b);$ 

if  $k_1 < w - 1$  then
     $x_1 \leftarrow \{m_1, a[w - 2 : 0]\};$ 
     $p \leftarrow 0;$ 
end
else
     $x_1 \leftarrow \{m_1, i_1, 1'b1\};$ 
     $p \leftarrow k_1 - (w - 2);$ 
end

if  $k_2 < w - 1$  then
     $x_2 \leftarrow \{m_2, b[w - 2 : 0]\};$ 
     $q \leftarrow 0;$ 
end
else
     $x_2 \leftarrow \{m_2, i_2, 1'b1\};$ 
     $q \leftarrow k_2 - (w - 2);$ 
end

 $temp \leftarrow booth\_mul(x_1, x_2);$  // Booth multiply
 $sum \leftarrow p + q;$  // Amount of shifts
 $R \leftarrow Barrel\_shifter(temp, sum);$ 
return  $R;$ 

```

, and

$$\begin{aligned} x_2 &= \{msb(b), b[k_2 : w - 2], 1'b1\} \\ &= \{1'b0, b[11 : 6], 1'b1\} = 01001011 \end{aligned}$$

The Booth multiplier then produces an intermediate product of width $2w$:

$$temp = Booth_mul(x_1, x_2) = x_1 \times x_2 = 1110001001101001$$

Since truncation alters the magnitude, correction is performed using a barrel shifter. The required shift is determined as follows:

$$p = \begin{cases} k_1 - (w - 2), & \text{if } k_1 \geq w - 1, \\ 0, & \text{otherwise,} \end{cases}$$

$$q = \begin{cases} k_2 - (w - 2), & \text{if } k_2 \geq w - 1, \\ 0, & \text{otherwise.} \end{cases}$$

The total shift value $p + q$ is supplied to the Barrel Shifter along with the Booth product $temp$, which is first sign-extended from $2w$ to $2n$ bits and then left-shifted by $p + q$ to restore the correct magnitude.

In the example of Fig. 14, the shift values are

$$p = k_1 - (w - 2) = 13 - 6 = 7, \quad q = k_2 - (w - 2) = 11 - 6 = 5.$$

Thus, the total shift is

$$sum = p + q = 7 + 5 = 12.$$

The barrel shifter then sign-extends the Booth product $temp$ from $2w$ to $2n = 32$ bits and performs a 12-bit left shift, thereby restoring the correct magnitude. The resulting value forms the final output of TR-ABM1.

In the example in Fig. 14

$$\begin{aligned} y &= sign_extend(temp) = \{16\{MSB(temp)\}, temp\} \\ &= 111111111111111111110001001101001 \end{aligned}$$

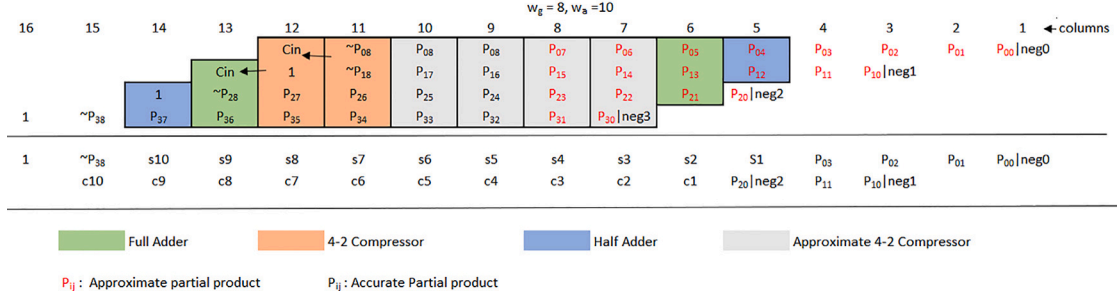


Fig. 15. Architecture of 8-bit ABM used in 16-bit TR-ABM2.

Algorithm 2: mux

Input: $select(k_1 \text{ or } K_2)$: $\log_2(n)$ -bit input, a : n -bit input
Output: out : $(w-2)$ -bit output

if $select < w - 1$ **then**
 $out \leftarrow a[w - 1 : 0]$; // Take lowest w bits
end
else
 $out \leftarrow a[select : w - 2]$;
 // Take $w-2$ bits starting from $select$
end
return out ;

Algorithm 3: Barrel_shifter

Input: $temp$: $2w$ -bit input, sum : $\log_2(n) + 1$ -bit input
Output: $result$: $2n$ -bit output

$y \leftarrow \{16\{MSB(temp)\}, temp\}$; // sign extent $temp$ to $2n$ bits
 $result \leftarrow y \ll sum$; // Left shift by sum
return $result$;

The output of TR-ABM1 is the output of barrel shifter, which is given by

$$R = y \ll (p + q) = y \ll 12$$

$$= 11111110001001101001000000000000$$

where the exact result of $a \times b$ is given by

$$a \times b = 1111111000100110110110110101010$$

As an additional illustration (not shown in Fig. 14), consider the case where an input operand to TR-ABM1 contains LO/LZ position less than $w - 1$. Let

$$b = 0000000000110010, \quad k_2 = 5.$$

Since $k_2 < w - 1$ ($w = 8$, so $w - 1 = 7$), the multiplexer selects the lowest w bits of b , giving

$$x_2 = b[w - 1 : 0] = 00110010,$$

and therefore

$$q = 0.$$

In this case, the truncated operand x_2 is multiplied with x_1 using the Booth multiplier. The total shift amount is

$$p + q = 7 + 0 = 7.$$

Accordingly, the intermediate product $temp$ is first sign-extended from width $2w$ to $2n$ bits and then shifted left by seven positions to produce the final output of the TR-ABM1.

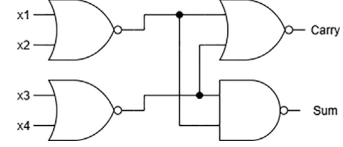


Fig. 16. Approximate Compressor (AC) [28].

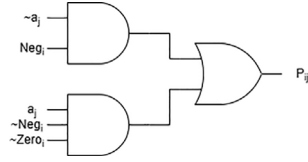


Fig. 17. Approximate Partial Product Generator (APG) proposed in [2].

Table 5 presents a comparison of the proposed TR-ABM1 architecture for different values of w against a previous design that employs LOZD along with an encoder for operand truncation [5]. The results demonstrate that TR-ABM1 achieves significant reductions in area, power, and delay relative to the prior work [5], while maintaining the same level of computational accuracy.

Furthermore, TR-ABM1 is evaluated against several other multipliers reported in the literature, with the comparative results summarized in Table 8. The detailed analysis corresponding to these results is provided in the section accompanying Table 8. An additional feature of TR-ABM1 is its runtime configurability through the parameter w , which provides flexibility for application-specific accuracy-efficiency trade-offs. The key advantage of the proposed architecture, however, lies in the use of the LOZPD module, which enables TR-ABM1 to achieve substantially lower Area-Delay Product (ADP) and Power-Delay Product (PDP) compared to prior designs, while preserving computational accuracy. Moreover, the modular structure of the architecture facilitates scalability to a wide range of operand widths and precision requirements.

2.6. Proposed truncated approximate booth Multiplier2 (TR-ABM2)

TR-ABM2 is derived from TR-ABM1 by substituting the conventional Booth multiplier with an approximate Booth multiplier (ABM). The proposed 8-bit ABM architecture, shown in Fig. 15, illustrates the adopted partial product generation and accumulation scheme. Gray cells indicate 4-2 Approximate Compressors (ACs) [28], while the red colored partial products in the least significant eight columns denote approximate partial products generated using the Approximate Partial Product Generator (APG) [2]. The selected AC and APG offer reduced hardware cost and balanced error characteristics, as their truth tables yield equal positive and negative errors, thereby minimizing the mean relative error. The AC truth table is given in Table 6, with the corresponding circuit shown in Fig. 16. In TR-ABM2, the truncation

Table 5
Comparison of proposed TR-ABM1 with TR-ABM in [5].

Truncated ABM	w	Area (μm^2)	Power (μW)	Delay (ps)	ADP ($\mu\text{m}^2\mu\text{s}$)	PDP (Wps)	MRED $\times 10^{-2}$	NMED $\times 10^{-4}$
TR-ABM1 Yuxuan [5]	6	2316.114 2864.11	183.002 283.74	2652 3520	6.142 10.081	0.485 0.998	2.93 2.93	70.40 70.40
TR-ABM1 Yuxuan [5]	8	2917.09 3734.54	279.71 418.47	3294 4184	9.608 15.625	0.921 1.751	0.7344 0.7344	17.15 17.15
TR-ABM1 Yuxuan [5]	10	3844.29 4627.68	430.72 545.85	3716 4496	14.285 20.8	1.60 2.454	0.182 0.182	4.416 4.416

Table 6

The truth table of the approximate 4:2 compressor.[28].

x_4	x_3	x_2	x_1	Carry	Sum	Error distance
0	0	0	0	0	0	0
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	1	-1
0	1	0	0	0	1	0
0	1	0	1	0	0	+1
0	1	1	0	1	1	+1
0	1	1	1	0	1	0
1	0	0	0	0	0	0
1	0	0	1	0	1	+1
1	0	1	0	1	1	+1
1	0	1	1	0	1	0
1	1	0	0	0	1	-1
1	1	0	1	1	1	0
1	1	1	0	1	1	0
1	1	1	1	1	1	-1

parameter is fixed at $w = 8$ bits, as its impact has already been investigated in the context of TR-ABM1. Instead, two additional configuration parameters, w_g and w_a , are examined in detail and are also illustrated in Fig. 15. Here, w_a denotes the number of columns in which the approximate 4-2 compressor (AC) is employed within the ABM, while w_g specifies the number of least significant columns where the partial products are generated by the APG. In the ABM design shown in Fig. 15, $w_g = 8$ indicates that APGs are applied to the 8 LSB columns of the partial product matrix, which are highlighted in red, and $w_a = 10$ means that ACs are employed up to LSB 10 columns, denoted by gray cells.

Truth Table for APG is given in Table 7. From the truth table, the equation for APG is given by:

$$p_{ij} = \overline{a_j} \cdot \text{neg}_i + a_j \cdot \overline{\text{neg}_i} \cdot \text{zero}_i. \quad (12)$$

The circuit for this APG is given in Fig. 17. The expressions for two_i , neg_i , zero_i are given by:

$$\text{two}_i = b_{2i+1} \cdot \overline{b_{2i}} \cdot \overline{b_{2i-1}} + \overline{b_{2i+1}} \cdot b_{2i} \cdot b_{2i-1} \quad (13)$$

$$\text{neg}_i = b_{2i+1} \cdot \overline{b_{2i-1}} + b_{2i+1} \cdot \overline{b_{2i}} \quad (14)$$

$$\text{zero}_i = \overline{b_{2i+1}} \cdot \overline{b_{2i}} \cdot \overline{b_{2i-1}} + b_{2i+1} \cdot b_{2i} \cdot b_{2i-1} \quad (15)$$

where these signals (two_i , neg_i , zero_i) represent the accurate values, and these accurate expressions are used in the proposed design.

The impact of varying w_g and w_a on error and hardware metrics is systematically evaluated and summarized in Table 8. The results are benchmarked against state-of-the-art approximate multipliers, as discussed in the following section. This configuration-driven approach enables a flexible trade-off between accuracy and efficiency, and the comparative analysis demonstrates that TR-ABM2 achieves superior area and power savings relative to existing designs.

3. Error analysis and VLSI implementation results

Error analysis of the proposed multipliers was carried out through Monte Carlo simulations using 10 million uniformly distributed random input vectors. To quantify the approximation quality, three widely used metrics were evaluated: Mean Relative Error Distance (MRED), Normalized Mean Error Distance (NMED), and Probability of Error Distance (PRED). Logic synthesis was performed in Cadence Genus[®], targeting a 90-nm CMOS technology node. The comparative results against state-of-the-art (SOTA) approximate multipliers are summarized in Table 8. Let P denote the accurate multiplication result and P' represent the output of an approximate multiplier. The error metrics are mathematically defined as follows: $NMED = \text{averageof}|P - P'|/(2^n - 1)^2$

$$MRED = \text{average}|P - P'|/P$$

$$PRED = \text{Probabilityof}(|P - P'|/P > 0.02)$$

$$WEIS = 0.3 \cdot NMED + 0.4 \cdot MRED + 0.3 \cdot PRED$$

To quantify overall accuracy degradation, we introduce the Weighted Error Impact Score (WEIS), defined as a linear combination of NMED, MRED, and PRED. A higher weight (0.4) is assigned to MRED, as it reflects the average relative error and is generally more representative of approximation quality compared to NMED or PRED.

By analyzing the last three columns of Table 8, it is evident that multipliers such as R4ABM1p14, R4ABM2p14 [29], Park [6], and ABM-M1 [2] achieve lower WEIS values, reflecting high accuracy; however, their reductions in Area-Delay Product (ADP) and Power-Delay Product (PDP) are modest compared to the proposed multipliers. In contrast, the proposed TR-ABM designs deliver significantly greater ADP and PDP reductions while maintaining competitive accuracy. Specifically, TR-ABM1-w8 belongs to the class of highly accurate multipliers, achieving the lowest ADP in this group and a comparatively lower PDP, thereby demonstrating a superior accuracy-performance trade-off. This improvement primarily stems from integrating the proposed LOZPD into the TR-ABM1 architecture.

On the other hand, multipliers such as DRUM [24], ASROBA and ROBA [25], HLR-BM2 [30], and ILM-AE [31] exhibit higher WEIS values, indicating lower accuracy, offering smaller ADP and PDP savings compared to the proposed designs. For instance, Yuxuan [5], which employs LOZD-based truncation, achieves similar accuracy but provides smaller ADP and PDP reductions than TR-ABM1, highlighting the advantage of LOZPD integration. Although ILM-AE [31] records the lowest PDP among prior works, its WEIS is significantly higher than that of all proposed multipliers, reflecting substantial accuracy loss without comparable ADP benefits.

The TR-ABM2 design further explores accuracy-hardware trade-offs through different input truncation configurations (w_g , w_a). Configurations such as (6, 7) and (7, 7) yield low WEIS values with substantial ADP and PDP reductions, while more aggressive settings such as (8, 10) achieve further ADP and PDP savings at the expense of accuracy. Within the proposed family, TR-ABM2 achieves higher accuracy than TR-ABM1-w6 and lower ADP/PDP than TR-ABM1-w8, enabled by its novel approximation strategy that integrates ABM into the truncated multiplier architecture.

Table 7

Truth table of the approximate partial product generator (APG) [2].

Input bit groups			Recorder signals			Approx. pp_{ij} for $a_j a_{j-1}$				Exact OP	Approx. OP	Error
b_{2i+1}	b_{2i}	b_{2i-1}	neg_i	two_i	$zero_i$	00	01	10	11			
0	0	0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	1	1	+1A	+1A	0
0	1	0	0	0	0	0	0	1	1	+1A	+1A	0
0	1	1	0	1	0	0	0	1	1	+2A	+1A	+1
1	0	0	1	1	0	1	1	0	0	-2A	-1A	-1
1	0	1	1	0	0	1	1	0	0	-1A	-1A	0
1	1	0	1	0	0	1	1	0	0	-1A	-1A	0
1	1	1	0	0	1	0	0	0	0	0	0	0

Table 8

Comparison of Proposed TR-ABM with SOTA Approximate multipliers.

Multiplier	Label	(w _g , w _a)	MRED $\times 10^{-1}$	NMED $\times 10^{-3}$	PRED	Area (μm^2)	Power (μW)	Delay (ps)	ADP ($\mu\text{m}^2 \text{ W}$)	PDP (mW ps)	ADP (%) Reduction	PDP (%) Reduction	WEIS
Exact-Radix4	ex	–	0	0	0	6833.29	337.32	3684	25.17	1242.68	–	–	0
TR-ABM1-w6	p1	–	0.293	7.040	0.5973	2316.11	183.002	2652	6.142	485.32	–75.59	–60.94	0.1900
TR-ABM1-w8	p2	–	0.0734	1.760	0.0171	2917.09	279.71	3294	9.608	921.1	–61.83	–25.88	0.00636
TR-ABM2	p3	(8,7)	0.1075	2.551	0.1339	2711.97	227.17	3109	8.431	706.27	–66.50	–43.16	0.0452
	p4	(7,7)	0.0940	2.248	0.0856	2808.85	238.84	3148	8.842	751.87	–64.87	–39.49	0.0301
	p5	(6,7)	0.0928	2.221	0.0813	2850.48	246.188	3176	9.053	781.89	–64.03	–37.08	0.0287
	p6	(8,8)	0.1302	3.040	0.2122	2672.6	219.21	3109	8.309	681.55	–66.99	–45.15	0.0697
	p7	(7,8)	0.1175	2.758	0.1692	2742.24	225.55	3148	8.632	710.03	–65.70	–42.86	0.0562
	p8	(6,8)	0.1202	2.821	0.1794	2783.8	228.92	3176	8.841	727.04	–64.87	–41.49	0.0594
	p9	(8,9)	0.2166	5.009	0.4347	2602.22	209.81	3160	–8.223	662.99	–67.33	–46.65	0.1405
	p10	(7,9)	0.1990	4.577	0.3973	2671.1	210.71	3210	8.574	676.37	–65.93	–45.57	0.1285
	p11	(6,9)	0.2005	4.614	0.4005	2712.7	216.02	3210	8.707	693.42	–65.41	–44.2	0.1295
	p12	(8,10)	0.3542	8.070	0.6007	2535.6	192.04	3130	7.936	601.08	–68.47	–51.63	0.1967
	p13	(7,10)	0.3440	7.818	0.5845	2605.2	196.8	3140	8.180	617.95	–67.5	–50.27	0.1914
	p14	(6,10)	0.3466	7.869	0.5876	2646.87	203.67	3178	8.411	647.26	–66.58	–47.91	0.1925
Yuxuan-w6 [5]	a	–	0.293	7.040	0.5973	2864.11	283.74	3520	10.081	998.76	–59.94	–19.62	0.1900
Yuxuan-w8 [5]	b	–	0.0734	1.760	0.0171	3734.54	418.47	4184	15.625	1750.87	–37.92	+40.89	0.00636
DRUM6-16-s [24]	c	–	15.042	375.913	0.8221	3223.63	317.933	5076	16.363	1613.82	–34.99	+29.86	0.961
ASROBA [25]	d	–	10.791	259.557	0.9969	3367.44	198.996	5138	17.302	1022.44	–31.25	–17.72	0.8085
ROBA [25]	e	–	10.795	259.57	0.997	4330.22	297.91	6377	27.613	1899.77	+9.7	+52.87	0.8087
HLR-BM2 [30]	f	–	4.861	125.126	0.9673	3804.93	196.81	3870	14.725	761.65	–41.49	–38.7	0.5221
R4ABM1p14[29]	g	–	0.00977	0.017	0.0048	6616.06	275.4	3525	23.321	970.785	–7.34	–21.87	0.001835
R4ABM1p20[29]	h	–	0.3532	0.7352	0.1254	5975.72	257.71	3525	21.06	908.42	–16.32	–26.89	0.0519
R4ABM2p14[29]	i	–	0.0237	0.0143	0.00677	6075.63	253.46	3446	20.93	873.42	–16.84	–29.71	0.002983
R4ABM2p20[29]	j	–	1.7621	0.8115	0.1709	5010.67	213.98	3446	17.26	737.37	–31.42	–40.66	0.1219
ILM-AE [31]	k	–	5.191	124.925	0.999	2531.07	87.444	3687	9.332	322.41	–62.92	–74.05	0.5448
Park [6]	l	(14,13)	0.0226	0.0146	0.00624	4680.75	228.213	3527	16.51	804.91	–34.4	–35.22	0.00278
Park [6]	m	(12,11)	0.0064	0.00667	0.00223	5237.83	257.08	3527	18.47	906.72	–26.61	–27.03	0.000927
Park [6]	n	(10,10)	0.00209	0.0045	0.00086	5675.32	280.08	3527	20.016	987.84	–20.47	–20.5	0.000343
ABM-M1 [2]	o	(16,0)	0.00771	0.0196	0.00464	4871.41	280.412	3834	18.68	1075.1	–25.78	–13.48	0.001706
ABM-M1 [2]	p	(14,0)	0.00257	0.0059	0.00141	5048.52	296.135	3834	19.356	1135.38	–23.09	–8.63	0.0005275
ABM-M1 [2]	q	(12,0)	0.00103	0.0030	0.00049	5198.38	309.85	3845	19.987	1191.37	–20.59	–2.03	0.0001891
He et al. [16]	r	–	0.0786	0.0344	0.01655	3929.82	298.44	2815	11.06	840.1	–56.05	–32.39	0.00812

Table 9

Comparison of TR-ABM1-w8 with He et al. [16].

Input range	Proposed TR-ABM1-w8				He et al. [16]			
	MRED $\times 10^{-1}$	NMED $\times 10^{-3}$	PRED	WEIS	MRED	NMED	PRED	WEIS
[–32768,32767]	0.0734	1.76	0.0171	8.59×10^{-3}	0.0786	0.0344	0.0165	8.104×10^{-3}
[–16384,16383]	0.0732	0.4415	0.01714	8.202×10^{-3}	0.3152	0.0374	0.0524	0.0283
[–8192,8191]	0.073	0.1105	0.0174	8.173×10^{-3}	0.8935	0.0453	0.1581	0.0831
[–4096,4095]	0.0721	0.0276	0.0171	8.022×10^{-3}	4.219	0.0486	0.4144	0.293
[–2048,2047]	0.0705	0.0068	0.0163	7.712×10^{-3}	11.628	0.055	0.8252	0.7126
[–1024,1024]	0.0681	0.0017	0.0147	7.134×10^{-3}	45.931	0.0584	0.9687	2.1278
[–512,511]	0.0612	0.0004	0.0101	5.478×10^{-3}	157.49	0.0653	0.9976	6.5988
[–256,255]	0.0457	0.0001	0.0057	3.538×10^{-3}	549.24	0.069	1.004	22.27
[–128,127]	0	0	0	0	1642.67	0.076	1.0089	66.009

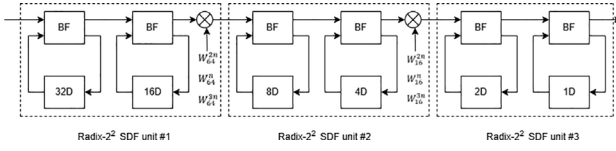


Fig. 18. 64-point FFT using radix-2² single-path delay feedback.

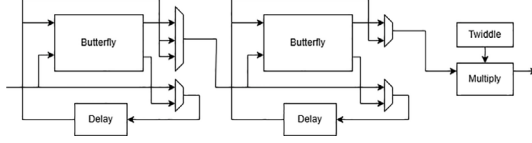


Fig. 19. Radix-2² Single Delay Feedback (SDF) unit.

3.1. Comparison with SOTA partial product truncated ABMs

Previous studies on approximate Booth multipliers have primarily relied on partial product truncation [16–19]. In these designs, the partial product array is split into an Accurate Part (AP) and a Truncation Part (TP), with nearly half of the partial products in TP being discarded. Various compensation circuits have been introduced to reduce the resulting error [16–19].

In contrast, the proposed multipliers adopt an operand truncation strategy that adaptively truncates inputs based on the position of their leading bit. This approach provides more accurate results across a wider range of inputs. When the leading bit position is smaller than the operand width, TR-ABM1 produces exact results. Table 9 compares the error distribution of TR-ABM1-w8 with the method of He et al. [16] under different input ranges. As observed, TR-ABM1-w8 achieves zero error for inputs within $[-128, 127]$, whereas partial product truncation methods incur higher errors for smaller input magnitudes due to their fixed truncation. Since neural network workloads often involve a large fraction of small weight values, the proposed multiplier is especially well-suited for such applications.

4. Application

To assess the efficiency and practicality of the proposed TR-ABM designs, we evaluate their performance on two computationally demanding applications: the Fast Fourier Transform (FFT) and image classification using Deep Neural Networks (DNN). Both applications are well-aligned with approximate computing paradigms due to their inherent tolerance to computational inaccuracies. FFT, a fundamental operation in signal processing, communications, and real-time analytics, can accommodate minor numerical errors without significantly affecting system performance [8,32]. Likewise, DNN-based image classification is resilient to moderate levels of approximation, with minimal impact on recognition accuracy [17,22,27,31,33]. For a fair and thorough evaluation, we replicate and benchmark our results against state-of-the-art designs, demonstrating that the proposed approach offers an effective trade-off between computational efficiency and accuracy.

4.1. FFT64 application

The 64-point Fast Fourier Transform (FFT) is implemented using a pipelined radix-2² architecture. As illustrated in Fig. 18, the design comprises multiple stages of radix-2² butterfly units, twiddle factor multipliers, and delay elements organized in a sequential pipeline. The FFT-64 consists of three stages ($\log_2 64 = 3$), where each stage performs radix-2² butterfly operations followed by multiplication with the corresponding twiddle factors. The pipelined structure ensures continuous data flow with reduced latency, while the Single Delay Feedback

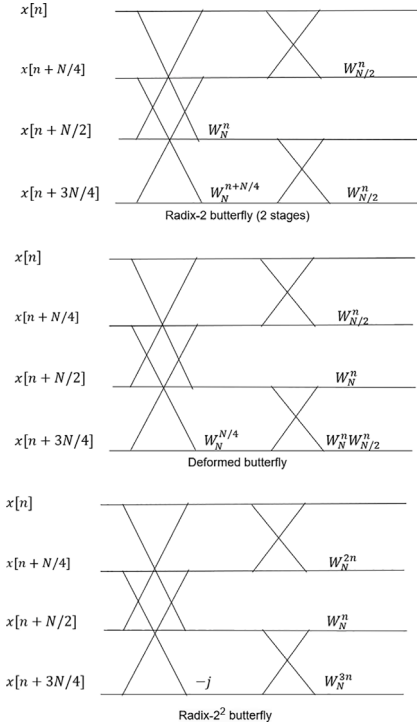


Fig. 20. Deriving radix-2² butterfly from radix-2 butterfly.

(SDF) mechanism (Fig. 19) minimizes storage requirements by reducing buffer size. The SDF unit efficiently reuses intermediate results while maintaining synchronization across stages, thereby improving computational efficiency. The radix-2² butterfly module, shown in Fig. 20, forms the core of the design, processing four input samples at a time through addition, subtraction, and twiddle factor multiplication. Each FFT stage applies these butterfly operations to progressively decompose the input signal into frequency components, as depicted in Fig. 18.

The FFT-64 design was validated through simulation in the Vivado environment. Predefined test inputs were applied, and the resulting FFT outputs were compared against exact computations. To assess the accuracy of the approximate FFT implementations, the outputs were post-processed using Python to compute error metrics, including Normalized Mean Error Distance (NMED), Mean Relative Error Distance (MRED), Number of Effective Bits (NoEB), and Probability of Relative Error Distance (PRED). These metrics were evaluated for a single set of 64 input samples. Compared to the large-scale evaluation presented in earlier sections (10 million random input cases), the reliability of these FFT-specific error metrics is relatively limited. NMED and MRED capture the average error magnitude, while NoEB estimates the effective precision loss due to approximation. The detailed error metrics of FFT implementations using each multiplier are summarized in Table 10.

For hardware evaluation, the FFT implementations were synthesized using the Cadence Genus tool with a 90 nm CMOS cell library, targeting a 20 ns clock period. The resulting area and power utilization are also reported in Table 10. Results indicate that FFT-64 implementations using the proposed multipliers achieve significant reductions in both area and power compared to state-of-the-art (SOTA) approximate multipliers. It is worth noting that no additional synthesis optimizations were applied; all multipliers – proposed and SOTA – were synthesized under identical conditions. However, the units for area and power reported in Table 8 differ in scale from those used in Table 10.

4.2. Neural network application

For handwritten digit recognition, the MNIST dataset is employed, consisting of 30,000 training images and 10,000 test images. A fully

Table 10
Performance of FFT64 system with the approximate multipliers.

Multiplier	(w _g ,w _a)	MRED	NMED × 10 ⁻⁶	NoEB	PRED	Area ($\mu\text{m}^2 \times 10^3$)	Power (mW)
Exact-Radix4	–	0	0	32	0	104.001	6.5426
TR-ABM1-6	–	0.4943	0.127	20.172	0.0664	68.054	4.1853
TR-ABM1-8	–	0.1578	0.1120	20.053	0.0390	74.886	4.7555
TR-ABM2	(8,7)	0.1640	0.1126	20.05	0.0703	70.743	4.4192
	(7,7)	0.1693	0.1128	20.05	0.0703	70.390	4.3877
	(6,7)	0.1692	0.1129	20.05	0.0664	70.743	4.4192
	(8,8)	0.1788	0.1138	20.055	0.0664	69.289	4.2972
	(7,8)	0.1757	0.1134	20.05	0.0703	69.845	4.3445
	(6,8)	0.1764	0.1135	20.05	0.0625	70.181	4.3785
	(8,9)	0.1941	0.1157	20.05	0.0703	68.757	4.2567
	(7,9)	0.2039	0.116	20.05	0.0664	69.335	4.2974
	(6,9)	0.2020	0.1378	20.27	0.0703	69.679	4.3217
	(8,10)	0.6854	0.1401	20.41	0.0703	68.246	4.2107
Yuxuan6[5]	(7,10)	0.3606	0.1169	20.84	0.0703	68.785	4.2549
	(6,10)	0.3600	0.0664	21.99	0.0742	69.154	4.2742
Yuxuan8[5]	–	0.4943	0.127	20.17	0.0664	69.786	4.2982
Yuxuan8[5]	–	0.1578	0.112	20.053	0.0390	76.368	4.8168
DRUM6-16-S [24]	–	2.849	0.282	20.42	0.0781	71.791	4.0048
ASROBA [25]	–	1.2862	0.2735	20.057	0.0664	75.143	4.2723
ROBA [25]	–	1.2935	0.274	20.055	0.0703	85.508	4.7093
HLR-BM2[30]	–	0.184	0.0736	21.3	0.0664	82.498	4.8347
R4ABM1p14[29]	–	0.0429	0.0001	31.14	0.0391	101.623	4.9729
R4ABM1p20[29]	–	0.4925	0.137	20.7	0.0703	96.797	4.7322
R4ABM2p14[29]	–	0.039	0.0001	31.36	0.039	99.322	5.1518
R4ABM2p20[29]	–	0.4933	0.1378	20.7	0.0703	91.662	4.9772
ILM-AE [31]	–	0.2441	0.0708	21.38	0.0507	66.507	3.7094
Park [6]	(14,13)	0.003515	0.001	31.4	0.0273	88.761	5.612
	(12,11)	0.00156	0.0001	31.678	0.0156	93.128	5.879
	(10,10)	0.000256	0.0001	31.752	0.0156	96.581	6.142
ABM-M1 [2]	(16,0)	0.000352	0.0001	31.529	0.0313	89.355	5.2492
	(14,0)	0.000234	0.00001	31.70	0.0234	90.816	5.3534
	(12,0)	0.0003906	0.00001	31.779	0.0039	92.064	5.4575

connected neural network is designed with an input layer of 784 neurons, three hidden layers containing 30, 30, and 10 neurons, respectively, and an output layer of 10 neurons ($784 \rightarrow 30 \rightarrow 30 \rightarrow 10 \rightarrow 10$). The network is trained for MNIST classification and achieves 96% accuracy after 30 epochs. Post-training, the weights and biases are quantized in a 16-bit fixed-point format for hardware realization. These quantized parameters are then used in a Verilog-based implementation of the network, tested using a Vivado testbench.

For hardware inference, the network employs the hardmax function [34] – which directly selects the maximum input – rather than the softmax function used during training. This choice, combined with the quantization from 64-bit floating-point to 16-bit fixed-point representation, results in a reduced accuracy of 91% when using exact 16-bit fixed-point multipliers. The classification accuracy for implementations using the proposed approximate multipliers, the exact multiplier, and state-of-the-art (SOTA) approximate multipliers is summarized in Table 11. The results demonstrate that the proposed approximate multipliers are highly suitable for neural network applications, as the accuracy degradation is minimal compared to both exact 16-bit multipliers and existing approximate designs.

5. Conclusion

The proposed Truncated Approximate Booth Multipliers (TR-ABMs) enhance energy efficiency, reduce area, and improve computational performance by employing Leading One/Zero Position Detectors (LOZPDs) for operand truncation. The energy- and area-efficient LOZPDs are particularly well-suited for signed truncated multipliers, as

they eliminate the pre- and post-processing modules typically required to convert unsigned multipliers into signed ones.

Two variants are proposed: TR-ABM1, which incorporates LOZPD-based operand truncation with a conventional Booth multiplier, and TR-ABM2, which replaces the exact Booth multiplier with an approximate counterpart. The degree of approximation is governed by design parameters such as multiplier width and the number of partial product columns implemented using Approximate Partial Product Generators (APGs) and Approximate Compressors (ACs).

Comprehensive evaluations – including Monte Carlo simulations with 10 million random inputs and synthesis on a 90 nm CMOS process – demonstrate that the proposed TR-ABMs deliver substantial improvements in Area-Delay Product (ADP) and Power-Delay Product (PDP) compared to state-of-the-art (SOTA) approximate multipliers. Specifically, TR-ABMs achieve 66.58%–75.59% ADP reduction and 47.91%–60.94% PDP reduction.

Application-level studies further validate the practicality of the proposed designs. In an MNIST classification network, the TR-ABMs achieve accuracy comparable to exact fixed-point Booth multipliers, confirming their suitability for AI hardware accelerators. Similarly, in a 64-point FFT (FFT64) implementation, the proposed multipliers yield significant area and power savings while preserving reliable accuracy.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Table 11
Accuracy of approximate multipliers on neural network.

Multiplier	(w,g,w,a)	Accuracy on DNN (%)
Exact-Radix4	–	91
TR-ABM1-6	–	90
TR-ABM1-8	–	90
TR-ABM2	(8,7)	90
	(7,7)	89
	(6,7)	89
	(8,8)	90
	(7,8)	89
	(6,8)	90
	(8,9)	89
	(7,9)	90
	(6,9)	90
	(8,10)	89
	(7,10)	90
	(6,10)	90
Yuxuan6[5]	–	90
Yuxuan8[5]	–	90
DRUM6-16-S [24]	–	8
ASROBA [25]	–	24
ROBA [25]	–	25
HLR-BM2[30]	–	18
R4ABM1p14[29]	–	91
R4ABM1p20[29]	–	51
R4ABM2p14[29]	–	89
R4ABM2p20[29]	–	15
ILM-AE [31]	–	24
Park [6]	(14,13)	89
	(12,11)	90
	(10,10)	90
ABM-M1 [2]	(16,0)	90
	(14,0)	90
	(12,0)	90
He <i>et al.</i> [16]	–	72

Data availability

No data was used for the research described in the article.

References

- [1] S. Nambi, U.A. Kumar, K. Radhakrishnan, M. Venkatesan, S.E. Ahmed, DeBAM: Decoder-based approximate multiplier for low power applications, *IEEE Embed. Syst. Lett.* 13 (4) (2020) 174–177.
- [2] S. Venkatachalam, E. Adams, H.J. Lee, S.B. Ko, Design and analysis of area and power efficient approximate booth multipliers, *IEEE Trans. Comput.* 68 (2019) 1697–1703, <http://dx.doi.org/10.1109/TC.2019.2926275>.
- [3] H. Jiang, J. Han, F. Qiao, F. Lombardi, Approximate radix-8 booth multipliers for low-power and high-performance operation, *IEEE Trans. Comput.* 65 (2016) 2638–2644, <http://dx.doi.org/10.1109/TC.2015.2493547>.
- [4] Z. Zhu, J. Huang, H. Zhang, H. Jiang, X. Gao, Error compensation based approximate booth multiplier with improved booth encoder and low complexity compressor, in: 2024 IEEE 7th International Conference on Electronic Information and Communication Technology, ICEICT, IEEE, 2024, pp. 424–429.
- [5] Y. Du, Z. Chen, B. Cheng, W. Shan, Design and analysis of leading one/zero detector based approximate multipliers, *Microelectron. J.* 136 (2023) 105783.
- [6] G. Park, J. Kung, Y. Lee, Simplified compressor and encoder designs for low-cost approximate radix-4 booth multiplier, *IEEE Trans. Circuits Syst. II: Express Briefs* 70 (2023) 1154–1158, <http://dx.doi.org/10.1109/TCSII.2022.3217696>.
- [7] A. Rafiq, S.M. Chaudhry, Design of an improved low-power and high-speed booth multiplier, *Circuits Systems Signal Process.* 40 (11) (2021) 5500–5532.
- [8] D.K.J. Rajanendiran, K. Priyadharsini, M. Ramkumar, *et al.*, Hybrid radix-16 booth encoding and rounding-based approximate karatsuba multiplier for fast Fourier transform computation in biomedical signal processing application, *Integration* 98 (2024) 102215.
- [9] B. G. Gowda, P. HC, M. VN, M. Rao, A probabilistic approach to design inexact compressors for approximate booth multipliers, *Analog Integr. Circuits Signal Process.* 122 (2) (2025) 1–23.
- [10] B. Boro, K.M. Reddy, Y.N. Kumar, M. Vasantha, Approximate radix-8 booth multiplier for low power and high speed applications, *Microelectron. J.* 101 (2020) 104816.
- [11] X. Zhu, H. Li, Y. Song, Exact and approximate radix-4 recoding multipliers for high-efficiency computation, *Microelectron. J.* (2025) 106579.
- [12] S.K. Beura, S.M. Mahanta, B.P. Devi, P. Saha, Inexact radix-4 booth multipliers based on new partial product generation scheme for image multiplication, *Integration* 94 (2024) 102096.
- [13] S. Hariprasad, D. Akriti, R. Rithvidas, M. Naveen, *et al.*, Image processing acceleration using radix-8 booth multiplier with custom designed 4: 2 compressors, in: 2024 5th International Conference on Data Intelligence and Cognitive Informatics, ICDICI, IEEE, 2024, pp. 1058–1063.
- [14] V. Gundavarapu, P. Gowtham, A.A. Angeline, P. Sasipriya, Design and evaluation of low power and area efficient approximate booth multipliers for error tolerant applications, *Microprocess. Microsyst.* 106 (2024) 105036.
- [15] W. Liu, L. Qian, C. Wang, H. Jiang, J. Han, F. Lombardi, Design of approximate radix-4 booth multipliers for error-tolerant computing, *IEEE Trans. Comput.* 66 (2017) 1435–1441, <http://dx.doi.org/10.1109/TC.2017.2672976>.
- [16] Y. He, X. Yi, Z. Zhang, B. Ma, Q. Li, A probabilistic prediction-based fixed-width booth multiplier for approximate computing, *IEEE Trans. Circuits Syst. I: Regul. Pap.* 67 (12) (2020) 4794–4803.
- [17] S.-N. Tang, Y.-S. Han, A high-accuracy hardware-efficient multiply-accumulate (mac) unit based on dual-mode truncation error compensation for cnns, *Ieee Access* 8 (2020) 214716–214731.
- [18] Z. Zhang, Y. He, A low-error energy-efficient fixed-width booth multiplier with sign-digit-based conditional probability estimation, *IEEE Trans. Circuits Syst. II: Express Briefs* 65 (2) (2017) 236–240.
- [19] Y.-H. Chen, Improvement of accuracy of fixed-width booth multipliers using data scaling technology, *IEEE Trans. Circuits Syst. II: Express Briefs* 68 (3) (2020) 1018–1022.
- [20] K.H. Abed, R.E. Siferd, CMOS VLSI implementation of a low-power logarithmic converter, *IEEE Trans. Comput.* 52 (11) (2003) 1421–1433.
- [21] J.N. Mitchell, Computer multiplication and division using binary logarithms, *IRE Trans. Electron. Comput.* (4) (2009) 512–517.
- [22] M.S. Kim, A.A. Del Barrio, L.T. Oliveira, R. Hermida, N. Bagherzadeh, Efficient Mitchell's approximate log multipliers for convolutional neural networks, *IEEE Trans. Comput.* 68 (5) (2018) 660–675.
- [23] S. Vahdat, M. Kamal, A. Afzali-Kusha, M. Pedram, Tosam: An energy-efficient truncation- and rounding-based scalable approximate multiplier, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 27 (2019) 1161–1173, <http://dx.doi.org/10.1109/TVLSI.2018.2890712>.
- [24] S. Hashemi, R.I. Bahar, S. Reda, DRUM: A dynamic range unbiased multiplier for approximate applications, in: 2015 IEEE/ACM International Conference on Computer-Aided Design, ICCAD, IEEE, 2015, pp. 418–425.
- [25] R. Zendegani, M. Kamal, M. Bahadori, A. Afzali-Kusha, M. Pedram, RoBA multiplier: A rounding-based approximate multiplier for high-speed yet energy-efficient digital signal processing, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 25 (2017) 393–401, <http://dx.doi.org/10.1109/TVLSI.2016.2587696>.
- [26] S. Vahdat, M. Kamal, A. Afzali-Kusha, M. Pedram, LETAM: A low energy truncation-based approximate multiplier, *Comput. Electr. Eng.* 63 (2017) 1–17.
- [27] S. Narayanamoorthy, H.A. Moghaddam, Z. Liu, T. Park, N.S. Kim, Energy-efficient approximate multiplication for digital signal processing and classification applications, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 23 (6) (2014) 1180–1184.
- [28] M. Ahmadinejad, M.H. Moaiyeri, F. Sabetzadeh, Energy and area efficient imprecise compressors for approximate multiplication at nanoscale, *AEU-Int. J. Electron. Commun.* 110 (2019) 152859.
- [29] W. Liu, L. Qian, C. Wang, H. Jiang, J. Han, F. Lombardi, Design of approximate radix-4 booth multipliers for error-tolerant computing, *IEEE Trans. Comput.* 66 (8) (2017) 1435–1441, <http://dx.doi.org/10.1109/TC.2017.2672976>.
- [30] H. Waris, C. Wang, W. Liu, Hybrid low radix encoding-based approximate booth multipliers, *IEEE Trans. Circuits Syst. II: Express Briefs* 67 (12) (2020) 3367–3371, <http://dx.doi.org/10.1109/TCSII.2020.2975094>.
- [31] M.S. Ansari, B.F. Cockburn, J. Han, An improved logarithmic multiplier for energy-efficient neural computing, *IEEE Trans. Comput.* 70 (4) (2021) 614–625, <http://dx.doi.org/10.1109/TC.2020.2992113>.
- [32] W. Liu, Q. Liao, F. Qiao, W. Xia, C. Wang, F. Lombardi, Approximate designs for fast Fourier transform (FFT) with application to speech recognition, *IEEE Trans. Circuits Syst. I: Regul. Pap.* 66 (12) (2019) 4727–4739.
- [33] G. Armeniakos, G. Zervakis, D. Soudris, J. Henkel, Hardware approximate techniques for deep neural network accelerators: A survey, *ACM Comput. Surv.* 55 (4) (2022) 1–36.
- [34] K. Vipin, Zynet: Automating deep neural network implementation on low-cost reconfigurable edge computing platforms, in: 2019 International Conference on Field-Programmable Technology, ICFPT, IEEE, 2019, pp. 323–326.